

TinyML-Based ECG Abnormality Detection for Resource-Constrained Embedded Systems

Mclawrence Okoror

Abstract— This work investigates a lightweight neural-network approach for binary ECG beat classification on resource-constrained embedded systems. Using the MIT-BIH Arrhythmia Database, 84,708 annotated beats were extracted and divided into source-aware training and testing groups, with normal beats classified against selected abnormal annotations. Reducing each beat from 180 to 90 raw ECG samples halved input storage while maintaining comparable performance. A compact 90–8–1 multilayer perceptron was selected, containing 737 trainable parameters and requiring 728 multiply-accumulate operations per inference. The floating-point model achieved 86.27% accuracy and 53.94% F1-score, while an integer-style implementation using INT8 weights and activations with INT32 accumulators achieved 86.28% accuracy and 53.98% F1-score. The quantized model required 1,500 bytes for the reported model constants and 130 bytes of inference working memory. The integer implementation reproduced the Python reference predictions for all 27,054 test vectors with zero mismatches. The model was subsequently deployed to an ESP32-S3 in Wokwi, where 100 selected test vectors also produced complete agreement with the reference implementation. These results demonstrate an end-to-end approach for implementing compact ECG classification models using integer-based inference on resource-constrained embedded platforms. The implementation code and supporting project files are publicly available at: <https://github.com/Mclawrence-Okoror/tinyml-ecg-classification>

Index Terms— ECG classification, TinyML, embedded machine learning, neural-network quantization, ESP32-S3.

I. INTRODUCTION

Electrocardiography (ECG) provides an important representation of cardiac electrical activity and is widely used for studying heart rhythm and arrhythmias. Automated ECG analysis has therefore remained an active research area, supported by annotated datasets such as the MIT-BIH Arrhythmia Database [1], [2]. The database contains 48 half-hour, two-channel ambulatory ECG recordings from 47 subjects sampled at 360 Hz and remains a widely used benchmark for arrhythmia classification.

ECG classification has evolved from methods based on ECG morphology and heartbeat intervals [3]–[5] to neural-network approaches, including one-dimensional convolutional and deep-learning models [6]–[10]. While these approaches can achieve strong classification

performance, their computational and memory requirements can make deployment on small embedded platforms challenging.

Tiny Machine Learning (TinyML) enables machine-learning inference on resource-constrained microcontrollers, where memory, computation, and power availability are limited [11], [12]. ECG-specific TinyML systems have demonstrated the feasibility of performing classification directly on embedded devices [13]. Quantization and reduced-precision inference can further reduce model storage and computational requirements while maintaining performance close to floating-point implementations [14]. Embedded inference frameworks such as TensorFlow Lite Micro have also demonstrated the deployment of machine-learning models on microcontrollers with limited resources [12].

Despite these developments, successful model training in a conventional computing environment does not guarantee identical behavior after conversion to an embedded implementation. Input representation, model size, numerical precision, and implementation consistency therefore remain important considerations for resource-constrained ECG classification.

This work presents an end-to-end approach for developing and deploying a compact ECG beat classifier for resource-constrained embedded systems. Using the MIT-BIH Arrhythmia Database, 84,708 annotated beats were extracted and formulated as a binary classification problem. A compact 90–8–1 multilayer perceptron was selected and converted to an integer-style inference pipeline using INT8 weights and activations with INT32 accumulators. The implementation was verified against the Python reference model and subsequently deployed to an ESP32-S3 target in Wokwi, covering ECG preprocessing, model reduction, quantization, embedded implementation, and cross-implementation validation.

II. METHOD

The methodology followed an end-to-end workflow from ECG data preparation to embedded implementation and cross-implementation validation. The process included beat extraction from the MIT-BIH Arrhythmia Database, source-aware dataset partitioning, input representation reduction, compact model development, integer quantization, C implementation, and deployment to an ESP32-S3 simulation target.

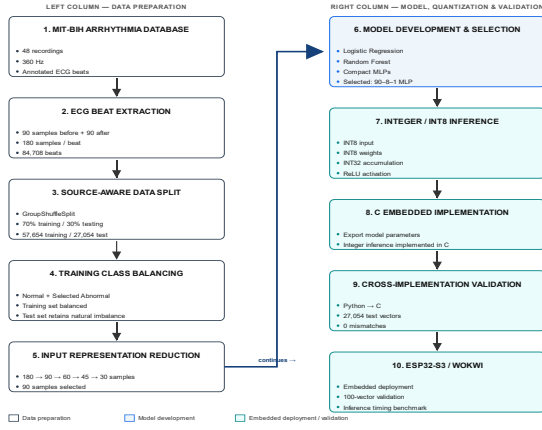


Fig. 1. Overall methodology pipeline.

A. Dataset and ECG Beat Extraction

The MIT-BIH Arrhythmia Database was used as the primary dataset. It contains 48 half-hour, two-channel ambulatory ECG recordings from 47 subjects, sampled at 360 Hz, with approximately 110,000 beat annotations [1], [2]. The recordings include records 100–124 and 200–234. Records 201 and 202 were treated as a single source group because they originated from the same analog tape.

For each annotated beat, 90 samples before and 90 samples after the annotation were extracted, producing a window of 180 samples. The resulting dataset contained 84,708 annotated beats, consisting of 75,033 normal beats and 9,675 selected abnormal beats. The classification task was formulated as a binary problem distinguishing normal beats from selected abnormal annotations.

The number of samples in each beat window was therefore given by

$$N_{samples} = 90 + 90 = 180 \quad (1)$$

At the sampling frequency of 360 Hz, the corresponding ECG window duration was

$$T_{window} = \frac{N_{samples}}{f_s} = \frac{180}{360} = 0.5 \text{ s} \quad (2)$$

A total of 84,708 annotated beats were extracted. The classification task was formulated as a binary problem consisting of normal beats and selected abnormal beats. Normal beats were represented by the N annotation, while the selected abnormal class consisted of the A and V annotations. The resulting dataset contained 75,033 normal beats and 9,675 selected abnormal beats. The percentage of each class was calculated as

$$P_c = \frac{N_c}{N_{total}} \times 100 \quad (3)$$

giving 88.58% normal beats and 11.42% selected abnormal beats. The resulting dataset composition is presented in Table I.

TABLE I. Dataset Composition

Class	Number of Beats	Percentage
Normal	75,033	88.58%
Selected abnormal	9,675	11.42%
Total	84,708	100.00%

B. Dataset Splitting and Class Balancing

To reduce the possibility of information leakage between training and testing data, recordings rather than individual beats were used as the grouping unit during dataset partitioning. This is particularly important for ECG classification because multiple beats originate from the same recording and patient, while the MIT-BIH database is organized around individual ECG recordings [1], [2]. A GroupShuffleSplit procedure was used with a test proportion of 30% and a random state of 42. The resulting partition contained 57,654 beats for training and 27,054 beats for testing. The total number of extracted beats can be verified as

$$N_{total} = N_{train} + N_{test} = 57,654 + 27,054 = 84,708 \quad (4)$$

The training set contained 50,622 normal beats and 7,032 selected abnormal beats, while the test set contained 24,411 normal beats and 2,643 selected abnormal beats. Because of the substantial class imbalance in the training data, only the training set was balanced. The normal class was randomly reduced to match the 7,032 selected abnormal beats, resulting in

$$N_{balanced} = 7,032 + 7,032 = 14,064 \quad (5)$$

The test set was not balanced and retained its natural class distribution so that model evaluation reflected the distribution of the held-out recordings. Patient- and recording-specific evaluation has also been considered in previous ECG classification studies [5], [6]. The resulting training and testing distributions are presented in Table II.

TABLE II. Training and Testing Distribution

Dataset	Normal	Selected Abnormal	Total
Training, before balancing	50,622	7,032	57,654
Training, after balancing	7,032	7,032	14,064
Testing	24,411	2,643	27,054

C. Input Representation Reduction

Input representation was investigated to determine whether the ECG beat representation could be reduced without substantially degrading classification

performance. Reducing input size is an important consideration for resource-constrained machine-learning systems because it can reduce both memory requirements and computational cost [11], [14]. The original 180-sample representation was compared with representations containing 90, 60, 45, and 30 samples. For the 180-sample representation, assuming 32-bit floating-point values, the storage requirement for a single input was

$$S_{180} = 180 \times 4 = 720 B \quad (6)$$

For the 90-sample representation, the corresponding storage requirement was

$$S_{90} = 90 \times 4 = 360 B \quad (7)$$

The reduction in input storage was therefore

$$R = \left(1 - \frac{S_{90}}{S_{180}}\right) \times 100 = 50\% \quad (8)$$

The same training and testing partitions were used for the different input representations, and a random forest baseline was used to compare their classification performance. The 90-sample representation was selected because it reduced the input storage requirement by 50% while maintaining performance comparable to the original 180-sample representation. The results are presented in Table III and Fig. 2.

TABLE III. Input Representation Reduction (Random Forest Baseline)

Samples per beat	F1-score (%)	Input storage (bytes)
180	48.73	720
90 (selected)	48.88	360
60	47.54	240
45	48.43	180
30	45.98	120

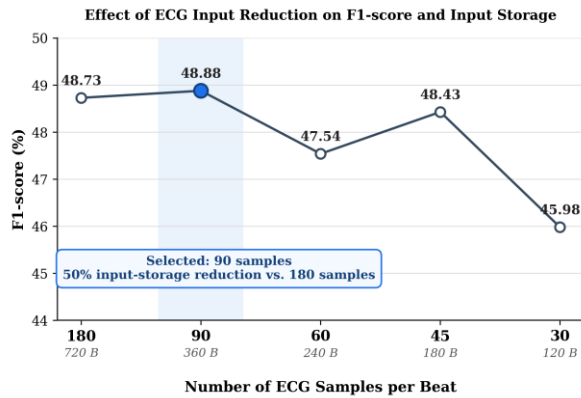


Fig. 2. Effect of input representation reduction on F1-score and input storage.

D. Model Development and Selection

Several machine-learning models were evaluated using the reduced 90-sample representation. Logistic

regression and random forest models were initially used as baseline approaches. Additional experiments investigated random forest tree-count reduction and compact multilayer perceptrons (MLPs). The evaluated MLP architectures included 90–16–8–1, 90–8–4–1, 90–8–1, 90–4–1, and 90–2–1, where the first value represents the input dimension, the intermediate values represent hidden-layer sizes, and the final value represents the single binary output. Compact neural-network architectures and reduced-precision implementations are commonly used when deploying machine-learning models under embedded resource constraints [14], [15].

Classification performance was evaluated using accuracy, precision, recall, and F1-score. Accuracy was calculated as

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (9)$$

Precision was calculated as

$$Precision = \frac{TP}{(TP + FP)} \quad (10)$$

Recall was calculated as

$$Recall = \frac{TP}{(TP + FN)} \quad (11)$$

and the F1-score was calculated as

$$F1 = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)} \quad (12)$$

Because the dataset was imbalanced, particular attention was given to precision, recall, and F1-score rather than accuracy alone. The 90–8–1 MLP was selected for subsequent embedded implementation because it achieved the highest F1-score among the evaluated MLP configurations while maintaining a substantially smaller parameter count than the larger neural-network configuration [14], [15]. The number of trainable parameters in the selected network was calculated as

$$N_{params} = (90 \times 8) + 8 + (8 \times 1) + 1 = 737 \quad (13)$$

The first layer contains 720 weights and 8 biases, while the second layer contains 8 weights and 1 bias, giving 737 trainable parameters in total.

The number of multiply-accumulate operations required for one inference was calculated as

$$N_{MAC} = (90 \times 8) + (8 \times 1) = 728 \quad (14)$$

The performance and storage characteristics of the evaluated models are summarized in Table IV. The selected 90–8–1 architecture is illustrated in Fig. 3.

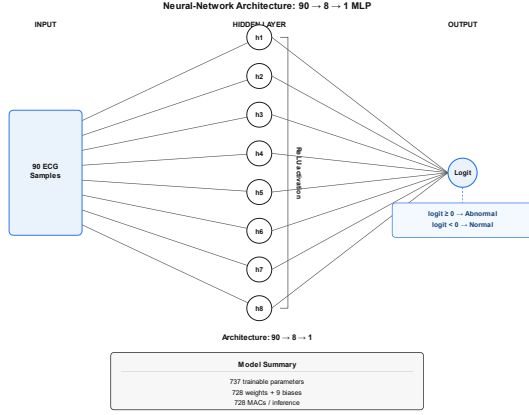


Fig. 3. Architecture of the selected 90-8-1 multilayer perceptron.

E. Quantization and Integer Inference

The selected floating-point MLP was converted to an integer-style inference pipeline to reduce numerical storage requirements and enable implementation using integer arithmetic on a resource-constrained embedded platform [14], [15]. The pipeline consisted of standardized input values, INT8 input quantization, INT8 weights, INT32 accumulation, rescaling, ReLU activation, INT8 hidden activations, and a final INT32 output accumulation. The original ECG input was first standardized using the training-set mean and standard deviation according to

$$x_{std} = \frac{(x - \mu)}{\sigma} \quad (15)$$

The standardized values were then quantized to signed 8-bit integers using a scale factor s :

$$q = \text{round}\left(\frac{x_{std}}{s}\right) \quad (16)$$

The corresponding approximate reconstructed value is

$$x_{std} \approx q \times s \quad (17)$$

The first layer used the quantized input and quantized weights. The products were accumulated using INT32 arithmetic. For hidden unit j , the integer accumulation can be represented as

$$a_j = \sum_{i=1}^{90} (q_{x,i} \times q_{w,ij}) + q_{b,j} \quad (18)$$

The accumulated value was rescaled before application of the ReLU activation:

$$h_j = \text{ReLU}(a_j \times s_{a1}) \quad (19)$$

The resulting hidden activation was quantized to INT8 using

$$q_{h,j} = \text{round}\left(\frac{h_j}{s_h}\right) \quad (20)$$

The second layer then accumulated the products of the quantized hidden activations and the quantized output weights:

$$a_o = \sum_{j=1}^8 (q_{h,j} \times q_{w2,j}) + q_{b,o} \quad (21)$$

The final binary prediction was obtained by applying a threshold to the output accumulator:

$$\hat{y} = \begin{cases} 1, & \text{if } a_o \geq 0 \\ 0, & \text{if } a_o < 0 \end{cases} \quad (22)$$

The complete integer-style inference pipeline is illustrated in Fig. 4.

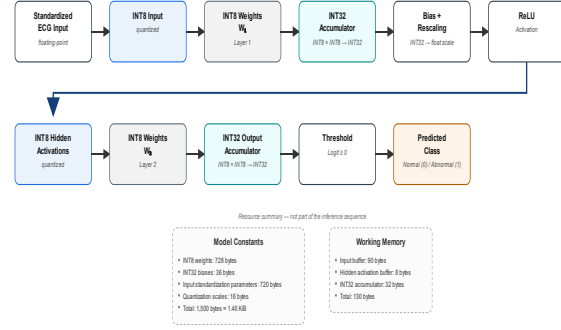


Fig. 4. Integer/INT8 inference pipeline.

The final implementation used INT8 weights and activations together with INT32 accumulators. The selected network contains 728 INT8 weight values, requiring 728 bytes, while its nine biases were represented using 32-bit integers, requiring 36 bytes. The reported input-standardization parameters required 720 bytes, while the quantization-scale parameters required 16 bytes. The resulting model constants were therefore

$$\begin{aligned} S_{model} &= 728 + 36 + 720 + 16 \\ &= 1,500 \text{ bytes} \\ &\approx 1.46 \text{ KiB} \end{aligned} \quad (23)$$

The inference working memory consisted of a 90-byte INT8 input buffer, an 8-byte hidden-layer buffer, and a 32-byte accumulator buffer, giving

$$S_{working} = 90 + 8 + 32 = 130 \text{ bytes} \quad (24)$$

The accumulator arithmetic was performed using signed 32-bit integers, whose representable range is

$$-2^{31} \leq a \leq 2^{31} - 1 \quad (25)$$

The final embedded resource requirements are summarized in Table V.

F. Embedded Implementation and Validation

The quantized network parameters and inference operations were implemented in C using the exported model parameters and quantization constants. To verify that the conversion from the Python reference implementation to the embedded-style C implementation did not alter the model's predictions, all 27,054 test vectors were evaluated by both implementations.

The agreement between the Python reference implementation and the C implementation was calculated as

$$\text{Agreement}_{\text{python-C}} = \frac{27,054}{27,054} \times 100 \quad (26)$$

$$= 100\%$$

No mismatches were observed across the complete test set. This result represents implementation agreement rather than classification accuracy; it demonstrates that the C implementation reproduced the predictions of the Python reference implementation for the evaluated test vectors. The C implementation was subsequently deployed to an ESP32-S3 target using the Wokwi simulation environment, consistent with the broader use of embedded platforms for local ECG classification [13]. A subset of 100 test vectors was used for cross-platform validation. The agreement between the C implementation and the ESP32-S3 simulation was

$$\text{Agreement}_{\text{C-ESP32}} = \left(\frac{100}{100} \right) \times 100 \quad (27)$$

$$= 100\%$$

All 100 selected vectors produced matching outputs between the reference implementation and the ESP32-S3 simulation. A final inference benchmark was also conducted in Wokwi using 1,000 repeated inference runs. The measured simulation time was 382.00 μs in total, corresponding to an average of approximately 0.38 μs per simulated inference and a throughput of approximately 2.62 million simulated inferences per second. These timing results describe the Wokwi simulation environment and should not be interpreted as measurements from a physical ESP32-S3 device. The complete Python-to-C-to-ESP32-S3 validation process is summarized in Fig. 5.

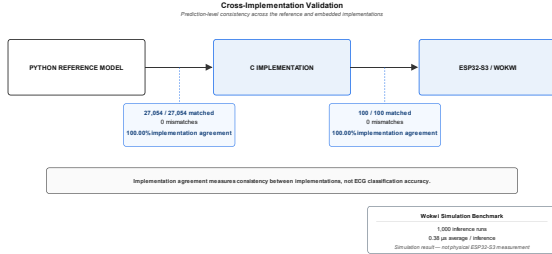


Fig. 5. Python-to-C-to-ESP32-S3 implementation validation.

III. EXPERIMENTAL RESULTS

A. Classification Performance

Table IV summarizes the performance of the evaluated models on the held-out test set. The 90–8–1 MLP achieved 86.27% accuracy and a 53.94% F1-score. Although the 90–4–1 model achieved slightly higher accuracy (86.50%), its F1-score was lower (52.91%). The 90–8–1 model was therefore selected for embedded

implementation because it provided the highest F1-score among the evaluated MLP configurations while maintaining a compact parameter count [14], [15].

Because the test set retained its imbalanced class distribution, accuracy alone does not fully describe abnormal-beat detection performance. Precision, recall, and F1-score were therefore considered alongside accuracy, consistent with evaluation practices in ECG classification studies [4]–[6].

TABLE IV. Performance comparison of evaluated models on the held-out test set.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Size (KB)
Logistic Regression	76.88	24.74	66.93	36.12	3.79
Random Forest (10 trees)	85.22	36.88	72.08	48.79	521.10
Decision Tree (depth 10)	82.78	31.90	67.16	43.25	26.48
MLP 90–16–8–1	81.40	32.84	86.49	47.61	30.42
MLP 90–8–4–1	82.99	34.89	85.58	49.57	20.68
MLP 90–8–1	86.27	40.12	82.29	53.94	20.11
MLP 90–4–1	86.50	40.13	77.64	52.91	16.45
MLP 90–2–1	85.99	39.06	77.41	51.92	13.83

B. Model and Resource Comparison

The comparison in Table IV demonstrates the trade-off between classification performance and model size. The selected 90–8–1 MLP contains 737 trainable parameters and requires 728 multiply-accumulate operations per inference. Its fully connected structure also simplifies conversion to an explicit integer-based inference implementation.

C. Quantization and Embedded Resource Usage

The floating-point and integer-style implementations produced nearly identical classification results. The floating-point model achieved 86.27% accuracy and a 53.94% F1-score, while the integer implementation achieved 86.28% accuracy and a 53.98% F1-score. The small differences indicate that the reported integer conversion did not materially change the observed classification performance.

As shown in Table V, the resulting model constants required 1,500 bytes (approximately 1.46 KiB), while the reported inference working memory was 130 bytes. Reduced-precision inference is commonly used to reduce memory and computational requirements for resource-constrained machine-learning deployment [14], [15].

TABLE V. Resource requirements of the integer-style inference implementation.

Component	Size / Requirement
Input dimension	90 INT8 values
Hidden layer	8 INT8 values
Output	1
Trainable parameters	737
INT8 weights	728 bytes
INT32 biases	36 bytes
Input-standardization parameters	720 bytes
Quantization-scale parameters	16 bytes
Model constants	1,500 bytes (≈ 1.46 KiB)
Input buffer	90 bytes
Hidden-layer buffer	8 bytes
Accumulator buffer	32 bytes
Inference working memory	130 bytes
MAC operations/inference	728

D. Cross-Implementation Validation

The Python and C implementations produced identical predictions for all 27,054 test vectors, corresponding to 100% implementation agreement. The ESP32-S3/Wokwi deployment also reproduced the reference predictions for all 100 selected vectors. These results demonstrate consistency between the reference model and embedded implementation rather than additional classification accuracy.

E. Embedded Deployment and Inference Timing

The integer inference implementation was deployed to an ESP32-S3 target using Wokwi. Across 1,000 repeated inference runs, the measured simulation time was 382.00 μ s, corresponding to approximately 0.38 μ s per simulated inference and 2.62 million simulated inferences per second. This result is specific to the Wokwi simulation environment and should not be interpreted as a physical-device benchmark. Embedded ECG classification on microcontroller-class platforms has also been demonstrated in previous work [13]. A representative real-ECG verification is shown in Fig. 6, where the Python reference prediction and ESP32-S3 prediction produced the same output.

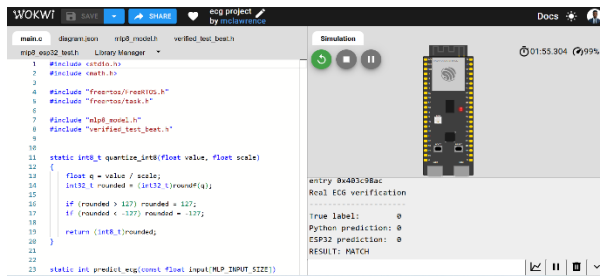


Fig. 6. ESP32-S3/Wokwi implementation showing the embedded inference code and real-ECG verification output.

F. Limitations

The evaluation was based on the MIT-BIH Arrhythmia Database and may not directly generalize to other datasets, patient populations, recording conditions, or acquisition hardware. The binary formulation also distinguishes normal beats from selected abnormal annotations rather than performing detailed multi-class arrhythmia classification. In addition, ESP32-S3 performance was evaluated in Wokwi rather than on physical hardware. Finally, the compact architecture prioritizes low resource requirements and implementation simplicity; larger or more specialized models may provide different performance at increased computational and memory cost.

Several additional limitations apply. The same fixed test partition was reused across the input-reduction, model-comparison, and architecture-selection experiments, so the reported test-set results should not be regarded as an untouched final holdout for every model-selection decision. The source-aware split reduces beat-level leakage between training and testing data, but it is not a fully stratified patient-wise validation protocol. Precision remained modest (40.12%) under the natural class imbalance, indicating a substantial number of false-positive detections. The reported 1,500 bytes represent model constants only and the 130 bytes represent inference working memory only; neither figure represents total firmware flash or total microcontroller memory usage. The Python-to-C and C-to-ESP32-S3 results measure implementation agreement rather than classification accuracy, and the ESP32-S3 comparison used a 100-vector subset. Lastly, no clinical validation was performed, and the system is a research prototype rather than a diagnostic device.

IV. CONCLUSION

This work presented an end-to-end TinyML approach for ECG abnormality detection on resource-constrained embedded systems. Using the MIT-BIH Arrhythmia Database, ECG beats were reduced from 180 to 90 samples and used to develop a compact 90–8–1 multilayer perceptron containing 737 trainable parameters and requiring 728 multiply-accumulate operations per inference.

The selected model achieved 86.27% accuracy and a 53.94% F1-score in floating-point form. Its integer-style implementation using INT8 weights and activations with INT32 accumulators achieved comparable performance, with 86.28% accuracy and a 53.98% F1-score. The resulting implementation required 1,500 bytes of reported model constants and 130 bytes of inference working memory.

Cross-implementation validation produced identical predictions between the Python reference and C

implementation for all 27,054 test vectors, while the ESP32-S3/Wokwi deployment reproduced the reference predictions for all 100 evaluated vectors. These results demonstrate that a compact neural-network model can be transformed into an integer-based embedded inference pipeline while preserving the computational behavior of the reference implementation.

Overall, the study demonstrates a practical workflow for developing, compressing, verifying, and deploying lightweight ECG classifiers for resource-constrained embedded platforms. Future work can investigate physical-device benchmarking, broader multi-class arrhythmia classification, and more efficient architectures or signal representations for improved robustness under real-world operating conditions.

REFERENCES

- [1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, May–June 2001.
- [2] G. B. Moody and R. G. Mark, "The MIT-BIH Arrhythmia Database on CD-ROM and software for use with it," in *Computers in Cardiology*, pp. 185–188, 1990, doi: 10.1109/CIC.1990.144205.
- [3] R. G. Mark, P. S. Schluter, G. B. Moody, P. H. Devlin, and D. Chernoff, "An annotated ECG database for evaluating arrhythmia detectors," *IEEE Transactions on Biomedical Engineering*, vol. 29, no. 8, p. 600, 1982.
- [4] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004, doi: 10.1109/TBME.2004.827359.
- [5] P. de Chazal and R. B. Reilly, "A patient-adapting heartbeat classifier using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 53, no. 12, pp. 2535–2543, 2006, doi: 10.1109/TBME.2006.883802.
- [6] T. İnce, S. Kiranyaz, and M. Gabbouj, "A generic and robust system for automated patient-specific classification of ECG signals," *IEEE Transactions on Biomedical Engineering*, vol. 56, no. 5, pp. 1415–1426, 2009, doi: 10.1109/TBME.2009.2013934.
- [7] S. Kiranyaz, T. İnce, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016, doi: 10.1109/TBME.2015.2468589.
- [8] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017, doi: 10.1016/j.combiomed.2017.08.022.
- [9] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, pp. 65–69, 2019, doi: 10.1038/s41591-018-0268-3.
- [10] A. H. Ribeiro *et al.*, "Automatic diagnosis of the 12-lead ECG using a deep neural network," *Nature Communications*, vol. 11, Art. no. 1760, 2020, doi: 10.1038/s41467-020-15432-4.
- [11] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," *Journal of King Saud University—Computer and Information Sciences*, vol. 34, no. 4, pp. 1595–1623, 2022, doi: 10.1016/j.jksuci.2021.11.019.
- [12] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.
- [13] E. Kim, J. Kim, J. Park, H. Ko, and Y. Kyung, "TinyML-Based Classification in an ECG Monitoring Embedded System," *Computers, Materials & Continua*, vol. 75, no. 1, pp. 1751–1764, 2023, doi: 10.32604/cmc.2023.031663.
- [14] C. R. Banbury *et al.*, "Benchmarking TinyML systems: Challenges and direction," *Proceedings of Machine Learning and Systems*, vol. 3, 2021.
- [15] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018, doi: 10.1109/CVPR.2018.00286.